Московский авиационный институт

(национальный исследовательский университет)

Институт №8 «Компьютерные науки и прикладная математика»

Кафедра №806 «Вычислительная математика и программирование»

Лабораторная работа №1 по курсу

«Операционные системы»

Вариант 3

Группа: М8О-209БВ-25

Студент: Апасов Д.К.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 31.10.2026

Москва, 2026

Постановка задачи

Вариант 3.

Родительский процесс создаёт один дочерний процесс. Первой строкой пользователь вводит имя выходного файла. Это имя передаётся дочерней программе как аргумент запуска. Родительский и дочерний процессы должны быть представлены отдельными программами, скомпилированными независимо друг от друга.

Родитель получает команды вида «число число число<перевод строки>» и передаёт их ребёнку через первый канал, связанный со стандартным вводом дочернего процесса. Второй канал используется для передачи данных от ребёнка родителю. Результаты ребёнок записывает в созданный им файл.

Дочерний процесс последовательно делит первое число строки на каждое последующее. Числа имеют тип int, их количество в строке может быть произвольным. Проверка деления на ноль выполняется в дочернем процессе. При обнаружении нулевого делителя дочерний и родительский процессы завершают работу.

Программа реализуется на языке Си для Linux. Необходимо предусмотреть обработку системных ошибок. Цель работы - освоить создание процессов и обмен данными между ними через каналы.

Общий метод и алгоритм решения

Использованные системные вызовы и функции:

- **fork()** - создаёт дочерний процесс.
- **pipe()** - создаёт неименованный канал; элемент [0] содержит дескриптор для чтения, элемент [1] - для записи.
- **dup2()** - подключает стандартный ввод и вывод ребёнка к соответствующим концам каналов.
- **execv()** - заменяет выполняемую ребёнком программу на отдельную программу child, сохраняя PID процесса.
- **open()** - открывает или создаёт файл результатов для записи.
- **read(), write()** - читают и записывают байты через дескрипторы.
- **close()** - закрывает файловый дескриптор.
- **waitpid()** - ожидает завершения ребёнка и получает его статус.
- **signal()** - задаёт игнорирование SIGPIPE, чтобы обрабатывать ошибку записи в закрытый канал.
- **exit()** - завершает процесс при ошибке внутри функции write_int().

Для обработки текста используются strchr() и strtoll(). Заголовки stdio.h, threads.h и stdatomic.h в исходных файлах не используются.

Общий метод и алгоритм решения (продолжение)

В работе используются две программы: parent.c и child.c. Родитель запускается пользователем, а программа ребёнка запускается через execv("./child", args).

Алгоритм родительской программы

- Прочитать имя выходного файла до перевода строки и завершить строку нулевым символом.
- Создать каналы parent_to_child и child_to_parent, затем вызвать fork().
- В дочерней ветке закрыть ненужные концы. Через dup2() связать дескриптор 0 с читающим концом первого канала, а дескриптор 1 - с записывающим концом второго. Закрыть исходные дескрипторы и выполнить execv(), передав имя файла.
- В родительской ветке закрыть ненужные концы и дождаться первого подтверждения от ребёнка после открытия файла.
- Передавать введённые байты в первый канал. После каждой строки ждать подтверждение по второму каналу.
- Завершить ввод при пустой строке или конце данных. Если последняя строка не закончена переводом строки, дописать его в канал.
- Закрыть оставшиеся концы каналов, вызвать waitpid() и проверить код завершения ребёнка.

Алгоритм дочерней программы

- Получить имя из argv[1], открыть файл с флагами O_WRONLY | O_CREAT | O_TRUNC и отправить родителю один байт подтверждения.
- Читать байты из стандартного ввода. Собирать текст одного числа в массив token до пробельного разделителя.
- Преобразовать текст через strtoll() и проверить диапазон int. Первое число сохранить в result, на каждое последующее делить result.
- При нулевом делителе или переполнении INT_MIN / -1 вывести сообщение в стандартный поток ошибок и завершиться с ошибкой.
- В конце строки записать результат функцией write_int(), отправить подтверждение родителю и начать обработку следующей строки.
- После получения конца данных закрыть файл и завершиться.

Подтверждение представляет собой один байт перевода строки. Если ребёнок завершился с ошибкой, родитель вместо подтверждения получает конец данных и также завершается. Лишние записывающие концы каналов закрываются, чтобы чтение могло корректно получить конец данных.

Деление целочисленное: дробная часть отбрасывается к нулю. Числа обрабатываются по одному, без хранения всей строки. Имя файла ограничено 1023 байтами, текст одного числа - 63 символами; число значений в строке фиксированным массивом не ограничено.

Код программы

parent.c

```
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>
#include <sys/wait.h>

int main(void) {
    const char p[] = "Enter filename: ";
    if (write(STDOUT_FILENO, p, sizeof(p) - 1) == -1)
        return EXIT_FAILURE;

    char filename_of_title[1024], c;
    int length_title_bytes = 0;
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, &c, 1)) == 1 && c != '\n') {
        if (length_title_bytes == 1023 || c == '\0')
            return EXIT_FAILURE;
        filename_of_title[length_title_bytes] = c;
        length_title_bytes++;
    }
    if (bytes == -1 || length_title_bytes == 0)
        return EXIT_FAILURE;
    filename_of_title[length_title_bytes] = '\0';

    int parent_to_child[2], child_to_parent[2];
    if (pipe(parent_to_child) == -1)
        return EXIT_FAILURE;
    if (pipe(child_to_parent) == -1)
        return EXIT_FAILURE;
    if (signal(SIGPIPE, SIG_IGN) == SIG_ERR)
        return EXIT_FAILURE;

    pid_t child = fork();
    if (child == -1)
        return EXIT_FAILURE;

    if (child == 0) {
        close(parent_to_child[1]);
        close(child_to_parent[0]);

        if (dup2(parent_to_child[0], STDIN_FILENO) == -1)
            return EXIT_FAILURE;
        if (dup2(child_to_parent[1], STDOUT_FILENO) == -1)
            return EXIT_FAILURE;

        close(parent_to_child[0]);
        close(child_to_parent[1]);
        char *args[] = {"./child", filename_of_title, NULL};
        execv("./child", args);
        return EXIT_FAILURE;
    } else {
        close(parent_to_child[0]);
        close(child_to_parent[1]);

        char ready;
        int success = read(child_to_parent[0], &ready, 1) == 1;
        int unfinished = 0;
```

parent.c (продолжение)

```
        while (success) {
            bytes = read(STDIN_FILENO, &c, 1);
            if (bytes == -1) {
                success = 0;
                break;
            }
            if (bytes == 0) {
                if (!unfinished)
                    break;
                c = '\n';
            } else if (c == '\n' && !unfinished) {
                break;
            }
            if (write(parent_to_child[1], &c, 1) != 1) {
                success = 0;
                break;
            }
            unfinished = c != '\n';
            if (c == '\n')
                success = read(child_to_parent[0], &ready, 1) == 1;
            if (bytes == 0)
                break;
        }

        close(parent_to_child[1]);
        close(child_to_parent[0]);

        int status;
        if (waitpid(child, &status, 0) == -1 ||
            !success || !WIFEXITED(status))
            return EXIT_FAILURE;

        return WEXITSTATUS(status);
    }
}
```

child.c

```
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <limits.h>

void write_int(int file, int value) {
    char buf[32], temp[32];
    int length = 0, count = 0;
    long long number = value;
    if (number < 0) {
        buf[length++] = '-';
        number = -number;
    }
    do {
        temp[count++] = (char)('0' + number % 10);
        number /= 10;
    } while (number > 0);
    while (count > 0)
        buf[length++] = temp[--count];
    buf[length++] = '\n';
    if (write(file, buf, length) != length) {
        const char msg[] = "error: failed to write result\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
}
```

child.c (продолжение)

```
int main(int argc, char **argv) {
    if (argc != 2)
        return EXIT_FAILURE;
    int file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file == -1) {
        const char msg[] = "error: failed to open file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        return EXIT_FAILURE;
    }
    if (write(STDOUT_FILENO, "\n", 1) != 1)
        return EXIT_FAILURE;

    char token[64], c;
    int length = 0, result = 0, first = 1;
    ssize_t bytes;
    while ((bytes = read(STDIN_FILENO, &c, 1)) == 1) {
        int separator = strchr(" \t\r\n\v\f", c) != NULL;
        if (c == '\0' || (length == 63 && !separator)) {
            const char msg[] = "error: invalid integer\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            return EXIT_FAILURE;
        }
        if (!separator) {
            token[length++] = c;
            continue;
        }
        if (length > 0) {
            token[length] = '\0';
            char *end;
            long long value = strtoll(token, &end, 10);
            if (end == token || *end != '\0' ||
                value < INT_MIN || value > INT_MAX) {
                const char msg[] = "error: invalid integer\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                return EXIT_FAILURE;
            }
            if (first) {
                result = (int)value;
                first = 0;
            } else if (value == 0) {
                const char msg[] = "error: division by zero\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                return EXIT_FAILURE;
            } else if (result == INT_MIN && value == -1) {
                const char msg[] = "error: integer overflow\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                return EXIT_FAILURE;
            } else {
                result /= (int)value;
            }
            length = 0;
        }
        if (c == '\n') {
            if (!first)
                write_int(file, result);
            if (write(STDOUT_FILENO, "\n", 1) != 1)
                return EXIT_FAILURE;
            first = 1;
        }
    }
    if (bytes < 0) {
        const char msg[] = "error: failed to read input\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        return EXIT_FAILURE;
    }
    if (close(file) == -1)
        return EXIT_FAILURE;
    return EXIT_SUCCESS;
}
```

Протокол работы программы

Программы компилируются отдельно и запускаются из одного каталога. Сборка с приведёнными параметрами завершилась без ошибок и предупреждений.

```
$ gcc -std=c11 -Wall -Wextra -Werror -pedantic parent.c -o parent
$ gcc -std=c11 -Wall -Wextra -Werror -pedantic child.c -o child
```

Тест 1. Последовательное деление целых чисел

Для завершения ввода после последней команды введена пустая строка. Обозначение в квадратных скобках поясняет действие пользователя.

```
$ ./parent
Enter filename: result.txt
100 5 2
-100 5 2
7 2
-7 2
0 5
[пустая строка]
$ echo $?
0
$ cat result.txt
10
-10
3
-3
0
```

Получены ожидаемые результаты: 100 / 5 / 2 = 10 и -100 / 5 / 2 = -10. Для 7 / 2 и -7 / 2 дробная часть отброшена к нулю. Нулевое первое число допускается: 0 / 5 = 0.

Тест 2. Деление на ноль

```
$ ./parent
Enter filename: zero.txt
20 2
10 0
error: division by zero
$ echo $?
1
$ cat zero.txt
10
```

Результат первой строки сохранён. При обработке второй строки ребёнок обнаружил нулевой делитель, и оба процесса завершились с ошибкой.

Вывод

В ходе лабораторной работы были изучены создание дочернего процесса с помощью fork(), запуск отдельной программы через execv(), обмен данными через неименованные каналы и переназначение стандартного ввода и вывода с помощью dup2().

Для варианта 3 реализованы две программы на языке Си. Родитель передаёт строки с целыми числами, а ребёнок выполняет последовательное деление и записывает результаты в файл. Проверка деления на ноль выполняется в ребёнке; при ошибке оба процесса завершаются. Правильное закрытие дескрипторов и ожидание через waitpid() обеспечивают завершение обмена и получение статуса ребёнка.